

Q1 Fundamentals

Process, PCB and Scheduling

OS: intermediary between users/applications and hardware; manages CPU, memory, files, I/O, protection and accounting

Program is passive code on disk; process is an active running program with address space and resources

PCB stores PID, state, PC, registers, scheduling info, memory info, open files and accounting info

States: new, ready, running, waiting/blocked, terminated

Ready queue holds processes ready for CPU; device queues hold processes waiting for I/O

Long-term scheduler admits jobs into memory and controls degree of multiprogramming

Medium-term scheduler swaps processes out/in to adjust memory pressure

Short-term scheduler chooses the next ready process for CPU

Dispatcher performs context switch and starts the chosen process

Context switch saves old PCB state and restores new PCB state; it is overhead and should be minimized in real-time systems

Process Operations and IPC

fork() creates a child process; parent receives child PID and child receives 0

exec() replaces the current program image while keeping the process identity

exit() terminates a process; parent can wait for child termination

Independent processes do not affect each other; cooperating processes share data or communicate

IPC by shared memory: fast but needs synchronization because the shared region is a critical section

IPC by message passing: send/receive through direct or indirect links/mailboxes

Blocking send/receive are synchronous; non-blocking send/receive are asynchronous

Buffers may be zero capacity, bounded capacity or unbounded capacity

Threads and Multithreading

Thread is the basic unit of CPU utilization: thread ID, PC, registers and stack

Threads in the same process share code, data, heap and files; each has its own PC/registers/stack

Benefits: responsiveness, resource sharing, economy and scalability

Concurrency interleaves tasks; parallelism runs tasks simultaneously on multiple cores

Amdahl's Law: speedup is limited by the serial part of a program

User threads are managed by a user library and switch fast; kernel threads are managed by the OS and can use multiple cores

Many-to-one: many user threads map to one kernel thread; one blocking call may block all

One-to-one: each user thread maps to one kernel thread; common in modern OS but has higher creation cost

Many-to-many: many user threads multiplexed over many kernel threads

Thread libraries: Pthreads, Win32, Java; implicit threading includes thread pool, OpenMP and GCD

Thread cancellation can be asynchronous or deferred; signal handling can be synchronous or asynchronous

Synchronization and Critical Section

Race condition: result depends on interleaving when processes update shared data

Critical section must satisfy mutual exclusion, progress and bounded waiting

Semaphore wait/P atomically decrements and may block; signal/V increments and may wake one waiter

Binary semaphore is mutex-like; counting semaphore represents multiple instances

Busy waiting spins while checking a condition; blocking waiting sleeps in a queue and saves CPU

Peterson uses flag[i] and turn; while(flag[j] && turn==j) protects mutual exclusion for two processes

Changing && to || may violate progress because a process can wait even when the other does not want to enter

Hardware support includes disabling interrupts, Test-and-Set, Compare-and-Swap and spinlocks

Classic patterns: bounded buffer, readers-writers, sleeping barber and dining philosophers

Deadlock

Deadlock requires four conditions: mutual exclusion, hold-and-wait, no preemption and circular wait

Prevention breaks at least one condition; avoidance checks safe state before allocation

Banker's Algorithm uses Available/Work, Max, Allocation, Need and Finish

Detection allows deadlock then detects cycles or no-progress states

Recovery may terminate processes or preempt resources, considering victim selection and rollback

CPU Scheduling

FCFS is non-preemptive and may cause convoy effect

SJF chooses shortest burst among arrived processes; SRTF is the preemptive version

RR uses time quantum; larger quantum reduces context switching and improves throughput but worsens response time

Priority scheduling may starve low-priority processes; aging can fix it

Multilevel queue separates classes; multilevel feedback queue lets jobs move between queues

Real-time RMS gives shorter period higher static priority

EDF chooses earliest absolute deadline and uses dynamic priority

Processor affinity keeps a thread on the same CPU to improve cache locality

Load balancing uses push migration or pull migration

Coarse-grained and fine-grained multithreading hide memory stalls differently

Memory Management

Logical address is generated by CPU; physical address is used by memory hardware; MMU translates addresses

Base register stores smallest legal physical address; limit register stores range size to prevent illegal access

Contiguous allocation can cause external fragmentation; fixed partitions cause internal fragmentation

First-fit picks first enough hole; best-fit picks smallest enough hole; worst-fit picks largest hole

Swapping moves processes between memory and disk; compaction can reduce external fragmentation

Paging divides logical memory into pages and physical memory into frames of equal size

Page number indexes page table; offset selects byte inside the frame

Page size is power of two; larger page size means smaller page table but more internal fragmentation

PTBR points to page table; PTLR stores page-table length

TLB caches page-table entries; hit saves one memory access

Valid/present bit distinguishes pages in RAM from pages on disk and triggers page fault when invalid

Page Tables and Virtual Memory

Hierarchical page tables reduce memory used by sparse address spaces

Hashed page tables hash virtual page numbers for large address spaces

Inverted page table has one entry per physical frame, saving memory but requiring search/hash

Shared pages improve memory efficiency by sharing read-only code

Demand paging loads pages only when referenced; pager controls page-in/page-out

Page replacement: FIFO, LRU, OPT, Clock/Second Chance

Thrashing happens when excessive page faults make the system spend most time paging

File System and Storage

File attributes include name, identifier, type, location, size, protection and time/user info

FCB/inode stores file metadata; open-file tables track currently opened files

Directories map names to files and support organization, search, sharing and protection

Mounting attaches a file system to the directory tree

Allocation: contiguous is fast random access but fragments; linked avoids external fragmentation but poor random access; indexed supports direct access with index overhead

Fixed-size blocks cause disk internal fragmentation in the last block of a file

Disk access time includes seek time, rotational latency and transfer time

Disk scheduling: FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK

Swap space may use a separate disk partition or file

RAID 0 stripes without redundancy; RAID 1 mirrors; RAID 2 uses Hamming-code style ECC; RAID 3/4 use dedicated parity; RAID 5 distributes parity; RAID 6 uses dual parity; RAID 10 mirrors then stripes

I/O, Protection, Security, VM

I/O hardware uses ports/registers; device drivers hide device-specific details

Polling repeatedly checks device status; interrupt-driven I/O lets CPU work until the device interrupts

DMA transfers data between device and memory with little CPU involvement

Blocking I/O waits; nonblocking I/O returns immediately

Kernel I/O subsystem handles scheduling, buffering, caching, spooling and error handling

I/O protection uses privileged instructions and user/kernel mode separation

Access matrix: subjects/domains as rows, objects as columns, rights in cells

RBAC assigns permissions to roles rather than individual users

Encryption protects confidentiality and supports compliance if data is stolen

Security levels include physical, human, operating-system and network security

Virtual machine is an isolated computer abstraction provided by a hypervisor/VMM; useful for isolation, consolidation, testing, snapshots and migration, but has overhead and management/security cost

More Q1 Items from Lecture Review

OS services often named in essays: user interface, program execution, I/O operations, file-system manipulation, communications, error detection, resource allocation, accounting and protection/security

System calls provide the interface from user programs to OS services; examples include process control, file management, device management, information maintenance, communication and protection

User mode cannot execute privileged instructions directly; kernel mode can access hardware and protected OS data structures

A trap or system call enters kernel mode; the OS validates parameters, performs the service and returns to user mode

Process creation may share all resources, share some resources or share no resources with the parent, depending on OS policy

Termination can be normal exit, error exit, fatal error or killed by another process

Orphan process is adopted by init/system process; zombie process has terminated but its parent has not collected its exit status

Cooperating processes are useful for information sharing, computation speedup, modularity and convenience

Thread states may include ready, running, suspended, terminated and destroyed; exact names depend on the thread library

Explicit threading is manually programmed; implicit threading lets compiler/runtime create/manage threads

Thread pool avoids repeated create/destroy cost and limits the number of active threads

OpenMP uses compiler directives for parallel regions; GCD dispatches blocks to system-managed queues

fork() in a multithreaded process may duplicate only the calling thread or all threads, depending on implementation; exec() replaces the process image

Dining philosophers illustrates deadlock/starvation in competing resource acquisition

A safe solution can impose resource ordering, allow at most $n-1$ philosophers, or use a monitor/semaphore protocol

Spinlocks are useful only for short waits on multiprocessors because the waiting CPU keeps executing

Disabling interrupts works on a single CPU but is not enough for multiprocessor synchronization

Deadlock prevention examples: request all resources at once to break hold-and-wait; impose total resource ordering to break circular wait; allow resource preemption when possible

Deadlock avoidance needs maximum demand information and chooses only allocations that keep the system safe

Deadlock detection with multiple resource instances uses Available, Allocation, Request and Finish, similar to Banker's no-progress check

Recovery by terminating all deadlocked processes is simple but costly; terminating one by one requires victim selection

Page-fault handling steps: trap to OS, check reference validity, find free frame or replace victim, read page from disk, update page table/TLB, restart instruction

Copy-on-write lets parent and child share pages after fork until one writes

Frame table records which physical frames are free or allocated

Working-set idea keeps the pages a process actively uses; too few frames can cause thrashing

Dirty/modify bit avoids writing unchanged victim pages back to disk

File access methods: sequential access reads in order; direct/random access jumps to fixed-size records; indexed access uses an index for lookup

For fixed-size records and random access, contiguous or indexed allocation is suitable; linked allocation is poor because it must follow pointers

NFS supports remote file access over a network and makes remote files appear like local files

File sharing needs permissions, locks and consistency rules

Protection can use access lists, capability lists or role-based permissions

Disk formatting creates sectors; boot block stores bootstrap code; bad-block management remaps unusable blocks

RAID 0+1 stripes first then mirrors the striped set; RAID 1+0 mirrors first then stripes mirrored pairs and is usually more fault-tolerant

I/O buffering handles speed mismatch; caching keeps frequently used data; spooling queues output for a device such as printer

Device driver converts generic OS I/O requests into device-specific commands

Distributed systems need communication protocols, naming, synchronization and fault tolerance

Extra Details for Short-Essay Answers

Internal fragmentation happens inside an allocated block or fixed partition; external fragmentation happens between allocated blocks as scattered holes

Internal fragmentation is predictable and can be reduced by smaller block/partition size; external fragmentation may require compaction or paging

The trade-off in page size: large pages reduce page-table entries and improve TLB coverage, but waste memory and may increase I/O of unused data

The trade-off in VM: strong isolation and easy migration versus overhead, resource contention and hypervisor attack surface

Access matrix can be implemented by access-control lists stored with each object, or capability lists stored with each subject/domain
 ACL is convenient for answering who can access this object;
 capability list is convenient for answering what this subject can access
 RBAC is suitable for organizations because permissions follow roles such as leader, member or guest

Encryption protects confidentiality, but key management is critical; if the key is lost, data may be unrecoverable

Directory structures can be single-level, two-level, tree-structured, acyclic graph or general graph

Tree directories are easy to search and organize; acyclic graph supports sharing without cycles

Hard links and symbolic links are common file-sharing mechanisms

File-system consistency may require journaling or recovery after crash

Disk scheduling trade-offs: FCFS is fair/simple but inefficient; SSTF reduces average seek but may starve far requests

SCAN/LOOK are more systematic and reduce starvation; LOOK stops at last request instead of disk end

C-SCAN/C-LOOK give more uniform waiting time by servicing in one direction

Always state the initial head position and direction; if direction is not given, state your assumption

Blocking I/O is easy to program but can waste process waiting time; nonblocking/asynchronous I/O improves overlap

Memory-mapped I/O maps device registers into address space; port-mapped I/O uses special I/O instructions

Interrupt handler must be short; long work is often deferred to kernel threads or later processing

DMA still needs CPU setup and final interrupt, but avoids continuous CPU copying

Protection domains define a set of objects and rights; a process may switch domains under controlled rules

Principle of least privilege: give each user/process only the rights needed to perform its task

Authentication verifies identity; authorization checks permissions; auditing records security-relevant actions

Network security tools include firewall, encryption and intrusion detection

Compact English Lines for Q1

OS services: The OS provides a controlled environment for program execution and resource sharing

System calls: A system call is the programming interface used by applications to request kernel services

Kernel mode: Kernel mode allows privileged operations; user mode protects the OS from user-program errors

PCB: The PCB is the data structure that lets the OS suspend and later resume a process correctly

Ready vs waiting: Ready means the process only needs CPU; waiting means it is blocked for I/O, resource or event

Preemption: Preemption removes a running process from CPU before it finishes, usually due to interrupt, time quantum or higher-priority arrival

Throughput vs response time: Larger RR quantum reduces context-switch overhead; smaller quantum improves interactive response

Starvation: Starvation occurs when a process waits indefinitely; aging gradually increases its priority

Shared memory: It is faster because processes communicate by reading/writing the same region, but it requires synchronization

Message passing: It is easier to protect and can work across machines, but copying and kernel mediation may add overhead

Direct communication names the receiver/sender explicitly; indirect communication uses mailbox/port

Zero-capacity buffer requires sender and receiver rendezvous; bounded buffer can block when full; unbounded buffer never blocks sender in theory

Monitor: A monitor combines shared data, procedures and synchronization; only one process is active in the monitor at a time
 Condition variable: wait releases the monitor lock and sleeps; signal wakes one waiting process

Mutex vs semaphore: A mutex is for mutual exclusion; a counting semaphore can represent multiple identical resources

Lost wakeup: If signaling and waiting are not synchronized, a thread may sleep even though the event already happened

Demand paging advantage: It allows large virtual address spaces and loads only needed pages

Demand paging disadvantage: First access to a missing page causes page fault and disk I/O

Frame allocation can be equal, proportional or priority-based

Global replacement chooses a victim from all processes; local replacement chooses only from the faulting process

Contiguous file allocation is suitable for fixed-size records and fast random access when file size is known

Linked allocation is good for sequential access and avoids external fragmentation, but pointer overhead and poor random access are weaknesses

Indexed allocation is flexible and supports random access, but index blocks consume space

Directory protection should specify read, write, execute, append, delete and list rights clearly

VM snapshot records the VM state for rollback; migration moves a running VM between physical hosts

Hypervisor Type 1 runs directly on hardware; Type 2 runs on top of a host OS

Distributed systems improve resource sharing and reliability but introduce network latency and consistency problems

Client-server and peer-to-peer are common distributed-system structures

Q2 Calculation Templates

CPU: $CT = \text{finish time}$; $TAT = CT - AT$; $WT = TAT - BT$; response time = first start - AT

Page faults: draw one column per reference, write F/H; FIFO hit does not reorder, LRU hit updates recency

TLB EAT without page fault = $h(TLB+M) + (1-h)(TLB+2M)$

Address bits: page offset bits = $\log_2(\text{page size})$; page number bits = $\log_2(\text{number of pages})$; frame bits = $\log_2(\text{number of frames})$

Disk movement = sum of absolute differences between consecutive head positions

Q3/Q4 Quick Templates

Banker: $\text{Need} = \text{Max} - \text{Allocation}$; $\text{Work} = \text{Available}$; if $\text{Need}[i] \leq \text{Work}$, P_i finishes and $\text{Work} += \text{Allocation}[i]$

Request: check $\text{Request} \leq \text{Need}$, $\text{Request} \leq \text{Available}$, then tentatively allocate and run safety algorithm

Q4: identify shared variable, identify semaphore protecting it, then explain race/deadlock/starvation if wait/signal is removed or reordered

Producer-consumer: $\text{wait}(\text{empty})$, $\text{wait}(\text{mutex})$, add, $\text{signal}(\text{mutex})$, $\text{signal}(\text{full})$; consumer uses full then empty

Readers-writers: mutex protects readcount; rw_mutex protects shared data; first reader locks writer out and last reader releases it

Q1 Fundamentals - Past-Paper Short Answers

Page Table / Present Bit

Question: How can the system distinguish between the pages that are in main memory from the pages that are on the disk? Explain your answer.

Answer: The page-table entry contains a valid/present bit. If it is present, the page is in a physical frame. If it is not present, the access causes a page fault; the OS loads the page from disk, updates the entry and restarts the instruction.

Process / Thread / Context Switch

Question: Compare process and thread. Why is switching threads less costly than switching processes? Which spends less time: switching two user-level threads in the same process, or in different processes?

Answer: A process owns an address space and resources. Threads in the same process share code, data, heap and open files. Same-process user-level thread switching is cheapest because only thread state such as PC, registers and stack pointer changes; process switching may change page tables, TLB and resource tables.

PCB / Queues / Schedulers

Question: Describe the ready queue and major activities of process management.

Answer: The ready queue stores processes that are ready to run but waiting for CPU time. Process management includes creating/deleting processes, suspending/resuming processes, CPU scheduling, synchronization, IPC and deadlock handling. PCB records the information needed to stop and later resume a process.

Memory Protection / Base-Limit

Question: How do base register and limit register help protect against illegal memory access? Why restrict access to memory?

Answer: The base register gives the lowest legal address and the limit register gives the valid range. Each memory reference is checked against them. This protects OS memory, protects one process from another, and prevents accidental or malicious modification of data/code.

Page / Frame / Page Size

Question: What is the difference between a page and a frame? Give two positive and two negative effects of increasing page size.

Answer: A page is a fixed-size block of virtual memory; a frame is a fixed-size block of physical memory. Larger pages reduce page-table size and may improve TLB reach. However, they increase internal fragmentation and may load more unused data, increasing memory waste or I/O cost.

Virtual Machine

Question: What is a virtual machine? Why is it useful for large-scale systems? Give arguments for and against it.

Answer: A VM is an isolated computer abstraction provided by a hypervisor/VMM and virtual hardware. It helps consolidation, isolation, testing, snapshots and migration. Arguments against it are performance overhead, resource contention, management complexity and possible hypervisor security risk.

File / Directory / Allocation / Fragmentation

Question: Briefly discuss the three major methods of allocating disk space. Why do file systems suffer from disk internal fragmentation? What are advantages of directories?

Answer: Contiguous allocation is fast and supports random access but suffers external fragmentation. Linked allocation avoids external fragmentation but random access is slow. Indexed allocation supports direct access but needs index blocks. Disk internal fragmentation occurs because files use fixed-size blocks and the last block may be partly unused. Directories organize files, improve searching, support sharing and protection.

Protection and Security

Question: Explain access matrix, RBAC, encryption advantages and file access protection for a project directory.

Answer: An access matrix lists subjects/domains as rows, objects as columns and allowed operations in cells. RBAC assigns permissions to roles. Encryption protects confidentiality and limits damage if data is stolen. For the project, leader gets read/write/execute/delete; members get read/write/execute but no delete; others get read/execute only.

I/O and DMA

Question: Describe why DMA is efficient. Compare polling and interrupt-driven I/O.

Answer: DMA transfers data directly between device and memory after CPU initialization, so the CPU is not busy copying every byte. Polling is simple and predictable but wastes CPU by repeated checking. Interrupt-driven I/O lets the CPU do other work, but interrupts add handling and context-switch overhead.

Deadlock and Waiting

Question: Describe deadlock prevention. What is busy waiting and what other waiting exists?

Answer: Deadlock prevention breaks one of the four necessary conditions: mutual exclusion, hold-and-wait, no preemption or circular wait. Busy waiting spins while repeatedly checking a condition; blocking waiting puts the process/thread to sleep in a queue until an event or signal wakes it.

Q2 Calculation - Complete Demonstrations

CPU Scheduling: FCFS / SJF / SRTF / Priority

Question: Processes P1(AT=0,BT=9), P2(1,5), P3(2,3), P4(3,4). Draw Gantt chart and calculate average turnaround and waiting time using FCFS or SJF. Another paper asks SRTF for P1(0,7), P2(2,4), P3(3,9), P4(5,10). Priority example: P1(0,3,3), P2(2,7,4), P3(3,5,1), P4(5,9,2), smaller number means higher priority.

Answer: FCFS: Gantt P1 0-9 | P2 9-14 | P3 14-17 | P4 17-21.
CT=(9,14,17,21), TAT=(9,13,15,18), WT=(0,8,12,14), avg TAT=13.75, avg WT=8.5. SJF non-preemptive: P1 0-9 | P3 9-12 | P4 12-16 | P2 16-21, avg TAT=13, avg WT=7.5. SRTF: P1 0-2 | P2 2-6 | P1 6-11 | P3 11-20 | P4 20-30; WT=(4,0,8,15), avg WT=6.75. Priority: P1 0-3 | P3 3-8 | P4 8-17 | P2 17-24; WT=(0,15,0,3), avg WT=4.5.

RMS / EDF Real-Time Scheduling

Question: RMS paper: P1(t=35,p=100), P2(t=10,p=50), P3(t=30,p=150). Calculate utilization and explain RMS. EDF paper: P1(AT=0,t=10,D=33), P2(4,3,D=28), P3(5,10,D=29). Calculate utilization and explain EDF.

Answer: RMS utilization = $35/100 + 10/50 + 30/150 = 0.75$. RMS assigns shorter period higher priority: $P2 > P1 > P3$, then draw a timing diagram by always running the available highest-priority periodic job. EDF uses dynamic priority; the job with the earliest absolute deadline runs first. For EDF data, utilization = $10/33 + 3/28 + 10/29 \approx 0.755$. In the answer, list absolute deadlines, preempt whenever a released job has an earlier deadline, then draw the timeline.

Page Replacement

Question: Calculate page faults. Examples: FIFO frame size 4 for 5 0 2 1 0 3 0 2 4 3 0 3 2 1 3 0 1 5; LRU frame size 3 for the same string; FIFO/LRU frame size 3 for 1 2 3 2 1 5 2 1 6 2 5 6 3 1 3 6 1 2 4 3; LRU frame size 3 for 1 2 3 2 6 3 4 1 5 6 1 6 4 2.

Answer: Write one column per reference and mark F/H. Results: FIFO(4) on the first string = 11 faults. LRU(3) on the first string = 13 faults. On the 20-reference string, FIFO(3)=14 faults and LRU(3)=11 faults. On 1 2 3 2 6 3 4 1 5 6 1 6 4 2 with LRU(3), faults = 10. For LRU, update recency on both faults and hits; for FIFO, hits do not change queue order.

TLB / Address Bits / Segmentation

Question: A TLB access takes 20 ns and memory access 50 ns; hit ratio 70%, no page fault. Another asks TLB=30 ns, memory=90 ns, hit ratio 70%, and asks hit ratio for EAT 130 ns. A system has 64 pages of 512 bytes and physical memory of 32 frames. Segment table: length/base: S1=(600,1000), S3=(900,1800), S4=(1000,2500); logical addresses (1,550),(3,908),(4,670).

Answer: $EAT = h(TLB+M) + (1-h)(TLB+2M)$. For 20/50/70%, $EAT = 0.7 \times 70 + 0.3 \times 120 = 85$ ns. For 30/90/70%, $EAT = 0.7 \times 120 + 0.3 \times 210 = 147$ ns; to get 130 ns, $130 = 210 - 90h$, so $h \approx 88.89\%$. Address bits: 64 pages need 6 page-number bits; 512-byte page needs 9 offset bits, so logical address =15 bits. 32 frames need 5 frame bits, so physical address =14 bits. Segmentation: (1,550) valid $\rightarrow$ 1550; (3,908) invalid because $908 > 900$; (4,670) valid $\rightarrow$ 3170.

Disk Scheduling and Memory Allocation

Question: SSTF examples: head 23, queue 54,97,73,128,15,44,110,34,45; head 28, queue 17,30,24,37,15,27,11,75,20,5. LOOK example: head 23 moving down, total 150 cylinders, queue 6,10,12,54,97,73,128,15,44,110,34,45. Best-fit example: jobs A=57K,B=920K,C=50K,D=701K; blocks 900K,910K,200K,300K.

Answer: SSTF head23: service 15,34,44,45,54,73,97,110,128; total movement 121. SSTF head28 with tie chosen by arrival order: 27,30,24,20,17,15,11,5,37,75; total 99. LOOK moving down: 15,12,10,6, then reverse to 34,44,45,54,73,97,110,128; total 139. Best-fit in arrival order: A(57K) $\rightarrow$ 200K block, B(920K) cannot fit, C(50K) $\rightarrow$ 300K block, D(701K) $\rightarrow$ 900K block; 910K remains unused.

Recent Q2 Pattern: SJF/SRTF, LOOK, Address Space

Question: Recent final-style questions ask both SJF and SRTF Gantt charts and turnaround time, LRU page replacement with frame size 3, LOOK disk scheduling, TLB EAT, and address calculation for 12-bit virtual address, 16-byte page size and 256-byte RAM. Recent resit-style questions ask RR with quantum=2, priority scheduling, FIFO frame size 4, SSTF disk scheduling and minimum page size for a single-level page table.

Answer: For SJF/SRTF, first sort events by arrival time. SJF chooses the shortest burst only when CPU becomes free; SRTF checks at every arrival and may preempt if the new job has shorter remaining time. TAT is CT-AT; WT is TAT-BT. For 12-bit VA and 16-byte page, offset bits=4 and page number bits=8, so virtual pages= $2^8=256$. For 256-byte RAM and 16-byte frames, physical frames= $256/16=16$. For RR(q=2), run each ready process for at most 2 units and put unfinished processes at the ready-queue tail. For single-level page table minimum page size, use the constraint that page table entries must fit the page/table space stated in the question.

Memory Fragmentation / Allocation Methods

Question: Three processes P1=19900, P2=19990, P3=19888 bytes are placed in fixed partitions of 20000 bytes. Is there fragmentation and can a process of 200 bytes be accommodated? Another paper uses P1=67000, P2=65000, P3=60000 in 70000-byte partitions and asks whether a 15000-byte process can be accommodated.

Answer: Fixed-size partitions create internal fragmentation inside each allocated partition. For 20000-byte partitions: leftover space = $100+10+112=222$ bytes, but it is split inside allocated partitions and cannot normally be combined for a 200-byte process; answer: internal fragmentation exists and the 200-byte process cannot be accommodated using that leftover internal space. For 70000-byte partitions: leftover = $3000+5000+10000=18000$ bytes, but it is internal fragmentation in separate partitions, so a 15000-byte process still needs a free partition and cannot use the combined leftover.

Q3 Banker's Algorithm - Full Answer

Full Matrix Example

Question: Consider Available(A,B,C,D)=(1,0,1,0). Max/Allocation: P0 Max(0,1,2,3) Alloc(0,1,1,2); P1 Max(2,0,0,2) Alloc(1,0,0,2); P2 Max(0,2,0,1) Alloc(0,1,0,1); P3 Max(2,2,0,0) Alloc(1,1,0,0); P4 Max(0,2,2,0) Alloc(0,1,2,0). Calculate Need, determine safe state, and decide whether P3 request (1,0,0,0) can be granted.

Answer: Need=Max-Allocation: P0(0,0,1,1), P1(1,0,0,0), P2(0,1,0,0), P3(1,1,0,0), P4(0,1,0,0). Start Work=(1,0,1,0). P1 can finish, Work=(2,0,1,2). P0 can finish, Work=(2,1,2,4). P2 can finish, Work=(2,2,2,5). P3 can finish, Work=(3,3,2,5). P4 can finish, Work=(3,4,4,5). Safe sequence: P1 $\rightarrow$ P0 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4, so the original state is safe. For request by P3: Request $\leq$ Need and Request $\leq$ Available, so tentatively allocate. New Available=(0,0,1,0), P3 Allocation=(2,1,0,0), P3 Need=(0,1,0,0). No unfinished process has Need $\leq$ Work=(0,0,1,0), so the new state is unsafe. The request cannot be granted immediately.

RAG / Request Template

Question: A paper asks to convert a Resource Allocation Graph to matrix representation and determine if there is deadlock. Another asks whether a request such as P2=(0,5,0) or P3=(0,1,0,0) can be safely granted.

Answer: For RAG: resource $\rightarrow$ process edges become Allocation; process $\rightarrow$ resource edges become Request/Need; Available = total instances - allocated instances. If every resource has one instance, a cycle implies deadlock; with multiple instances, a cycle is not sufficient, so run the safety/detection procedure. For request questions, first check Request $\leq$ Need, then Request $\leq$ Available, then tentatively allocate and run Banker's safety algorithm. Grant only if a safe sequence still exists.

Q4 Operating System in C - Expanded Demonstrations

Readers-Writers

Question: Readers-writers code uses mutex=1, rw_mutex=1, readcount=0. Reader increments readcount under mutex; first reader waits rw_mutex; last reader signals rw_mutex. Questions: indicate limitation; what if mutex lines are removed; what if only rw_mutex is used by readers?

Answer: Limitation: the classic readers-preference solution can starve writers if readers keep arriving. If mutex is removed, updates to readcount are no longer atomic; two readers may both see the wrong value, causing rw_mutex to be missed or released too early, so writer/reader mutual exclusion can fail. If only rw_mutex is used by readers, correctness for writers is preserved, but only one reader can read at a time, so the main advantage of allowing multiple concurrent readers is lost.

Producer-Consumer

Question: Bounded buffer uses empty=N, full=0, mutex=1. Producer: wait(empty), wait(mutex), add item, signal(mutex), signal(full). Consumer: wait(full), wait(mutex), remove item, signal(mutex), signal(empty). Questions: remove full; swap wait(empty) and wait(mutex) in producer; swap signal(full) and signal(empty).

Answer: Removing full lets a consumer consume when no item exists, causing buffer underflow and invalid data. If producer waits mutex before empty, it may hold mutex while blocked on a full buffer; consumers need mutex to remove an item and signal empty, so deadlock may occur. Swapping signal(full) and signal(empty) corrupts the resource counters: producers may think empty slots are available after producing, and consumers may not be notified that items exist.

Sleeping Barber

Question: Barber code uses waiting, CHAIRS=N, customers=0, barbers=0, mutex=1. Questions: what semaphores are used and what do their initial values mean; what happens if waiting-- and waiting++ are placed after V(mutex); what happens if CHAIRS=0?

Answer: customers counts waiting customers; initial 0 means the barber sleeps until a customer arrives. barbers counts available barber service; initial 0 means customers wait until barber is ready. mutex=1 protects waiting and chair checks. If waiting++/waiting-- happen after releasing mutex, waiting is updated outside the critical section, causing race conditions and wrong chair counts. If CHAIRS=0, no customer can wait; customers leave unless a direct handoff is specially implemented.

Opposite Semaphore Order / Peterson

Question: Two processes share semaphores $S=1$ and $R=1$. P1 waits S then R; P2 waits R then S. Can they be blocked forever? Can indefinite postponement occur? Peterson's algorithm changes `while(flag[j] && turn==j)` to `while(flag[j] || turn==j)`. Which property is violated?

Answer: Deadlock can occur: P1 executes `wait(S)`, P2 executes `wait(R)`, then P1 waits R and P2 waits S; both wait forever. Indefinite postponement is also possible under unfair scheduling if one process repeatedly gets resources while the other keeps waiting. For Peterson, changing `&&` to `||` violates progress because a process may wait even when the other process is not interested in entering the critical section.

Pipe and fork

Question: A POSIX program creates `pipe(fd)`, forks, child closes `fd[1]` and reads from `fd[0]`, parent closes `fd[0]` and writes 'Hello, pipe!' to `fd[1]`. Questions: what is the program attempting to do; what is the output; how to support child sending messages to parent at the same time?

Answer: The program creates one-way parent-to-child communication through a pipe. Expected output order may vary, but parent prints that it wrote the message, and child prints that it read the bytes and the string Hello, pipe!. To support simultaneous two-way communication, create two pipes: one parent-to-child and one child-to-parent. Each process closes unused ends and uses one pipe for writing and the other for reading.

Thread Order

Question: Some Q4 papers use pthreads and semaphores to force a required output order, or ask what happens if wait/signal operations are removed or reversed.

Answer: State the role of each semaphore first. A semaphore initialized to 0 usually blocks a thread until another thread signals it; a semaphore initialized to 1 often works as a mutex. If a wait is removed, the required order may be lost and output can appear in multiple interleavings. If a signal is removed, another thread may block forever. If waits are taken in inconsistent order, circular wait can occur and cause deadlock.

Ticket Booking / Shared Counter

Question: A common code scenario has multiple threads booking tickets or seats using a shared variable such as seats or availableTickets. Explain the bug if the lock is removed and how to fix it.

Answer: The shared counter must be protected by a mutex or binary semaphore. Without mutual exclusion, two threads may both read the same old value, both decide a seat is available, and both decrement or sell the same seat, causing lost update or overselling. The fix is to put the check and update in one critical section: `wait(mutex)`, `check seats>0`, `update seats`, then `signal(mutex)`. The lock should not be held during slow unrelated work such as printing or network I/O if the shared state has already been updated safely.

Q4 Checklist

Question: For any C synchronization question, what should be written before discussing the specific bug?

Answer: First identify the shared variable, such as readcount, waiting, x, seats or buffer. Second identify the semaphore or lock that protects it. Third state the invariant: buffer count must not go below 0 or above N; readcount must match active readers; waiting must not exceed CHAIRS; a shared counter must be updated atomically. Then explain what operation breaks that invariant and name the result: race condition, deadlock, starvation, lost update, underflow or overflow.

Semaphore Values

Question: Explain the meaning of common initial semaphore values in code questions.

Answer: A semaphore initialized to 1 usually means one thread may enter a critical section, so it acts like a mutex. A semaphore initialized to 0 usually means the thread must wait for another thread to signal an event. A semaphore initialized to N usually counts N identical resources, such as empty buffer slots or waiting chairs. In answers, connect the initial value to the resource count, not only to the syntax.

Common Modification Questions

Question: What should you say when a question asks what happens if a wait/signal is deleted, swapped, or moved outside a mutex?

Answer: If wait is deleted, a thread may enter before the condition is true. If signal is deleted, another thread may never wake up. If wait operations are acquired in different orders, circular wait can cause deadlock. If an update is moved outside mutex, the shared variable can be read and written concurrently, causing race condition. If signal is done too early, another thread may observe inconsistent shared state.